

Vibe Data Modeling

A Databricks White Paper

Amr Ali, Lead Solutions Architect, Databricks.

The reference implementation lives at github.com/databricks/vibe-modelling-agent. A short announcement blog accompanies this paper.

The problem with Data Modelling

For three decades, enterprise data modeling has followed the same rhythm.

A team of specialists gathers requirements. They interview stakeholders. They review systems of record. They sketch entity-relationship diagrams on whiteboards. They argue about naming. They draft a logical model. They review it. They draft a physical model. They review it again. They hand it to an ETL team. The first real data lands. Half the assumptions turn out to be wrong. The cycle starts over.

A solid analytical data model, the kind that actually runs production analytics and machine learning, has historically taken between six months and three years to build.

There is a shortcut. The industry data model. These are published schemas, often hundreds of tables deep, that a vendor or standards body sells as a shortcut for an entire sector. TM Forum has one for telecoms. ACORD has one for insurance. FHIR and HL7 for healthcare. ARTS for retail.

The pitch is compelling. Buy the template, save a year of work, start loading data on day one. The reality rarely matches the pitch.

An industry data model is the average of every business in that sector. It does not know about your product lines, your geographies, your regulatory footprint, your legacy constraints, your naming customs, or your organizational shape.

You inherit hundreds of tables you will never populate. You inherit naming conventions that do not match your terminology. You inherit relationship directions your workloads do not need. You spend months trimming, renaming, and rewiring the thing, which is most of the work you were trying to avoid in the first place.

This paper introduces a different approach. We call it vibe data modeling, because the loop at its heart is conversational.

You describe your business in natural language. The system builds a first complete model from that description. You look at what it built. You tell it what to change, again in natural language. It produces a new version.

Every version is fully linked, fully named, fully validated against more than two hundred enforceable rules, and fully deployed to the Databricks lakehouse if you want it to be. What used to take six months to three years now takes between a few hours and a few days.

The rest of this paper is about how that works, why the result is a business data model rather than an industry data model, and where the approach is honestly strong or weak.

Where the Model Lives: the Silver Layer

Before we go further, it is worth saying plainly what a vibe model is for.

In a Databricks medallion architecture, data flows through three layers. **Bronze** holds raw events as they arrive. **Silver** holds the conformed, normalized, governed analytical data model that every analyst, BI tool, and data science workload reads from. **Gold** holds derived products built on top of Silver: KPI summaries, feature tables, aggregated views, and metrics.

Vibe Data Modeling produces the Silver layer.

It is not a Gold layer tool. It does not generate pre-calculated metrics like `customer_churn_score` or `monthly_revenue_summary`. Those are derived insights that the analytics layer computes from the atomic tables the agent builds. In fact, the agent's rules actively reject pre-calculated metrics at design time (rule G12-R003), because they belong elsewhere.

It is not a Bronze layer tool either. It does not handle raw ingestion, change-data-capture, or schema evolution from upstream systems. Those concerns are handled by Lakeflow, Auto Loader, and the ingestion layer below.

It is the Silver layer. The analytical foundation. The place where the business's concepts become trustworthy tables.



Every other feature of the agent, from the four-level hierarchy, to the DAG (Directed Acyclic Graph) enforcement, to the SSOT (Single Source of Truth) rules, to the two architect reviews, to the metric-view generation, exists to produce a Silver layer that is worth trusting.

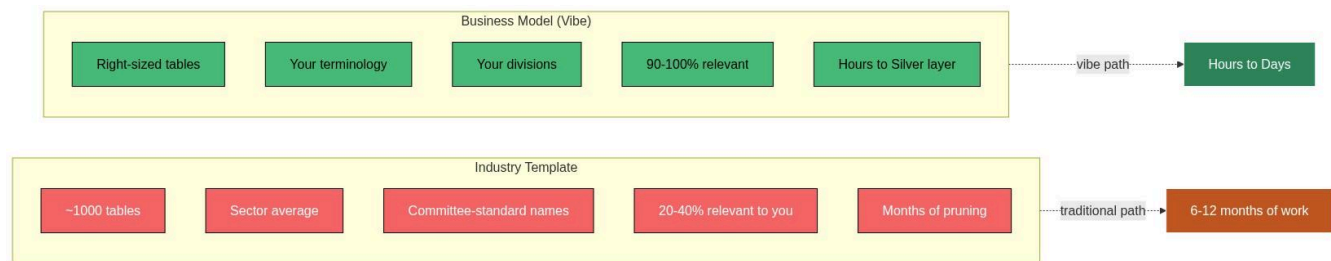
Industry Data Models vs Business Data Models

The difference between these two things is the whole reason vibe modeling exists. It is worth stating plainly before we go any further.

An industry data model is a template. Someone tried to capture the general shape of an entire sector in one schema. The intent is noble. The practical outcome is that any specific company in that sector uses maybe twenty to forty percent of the tables and spends months adapting the rest.

A business data model is not a template. It is a model of one organization, shaped by the actual operations of that organization, using terms that organization uses, with domains that match that organization's structure, and with relationships that match how that organization actually moves data around.

Vibe modeling produces the second kind.



	Industry Data Model	Business Data Model (Vibe)
Scope	An entire sector	One specific business
Customization	Manual pruning after delivery	Built in from the natural-language input
Relevance	20-40 percent directly applicable	90-100 percent directly applicable
Time to a usable Silver layer	Months of adaptation	Hours, end to end
Naming	Committee-standard terms	The terms your people actually use
Evolution	Each new version is a re-adoption event	Each new version is an iteration on the previous
Cost	License plus the labor of trimming	Compute cost of the LLM generation

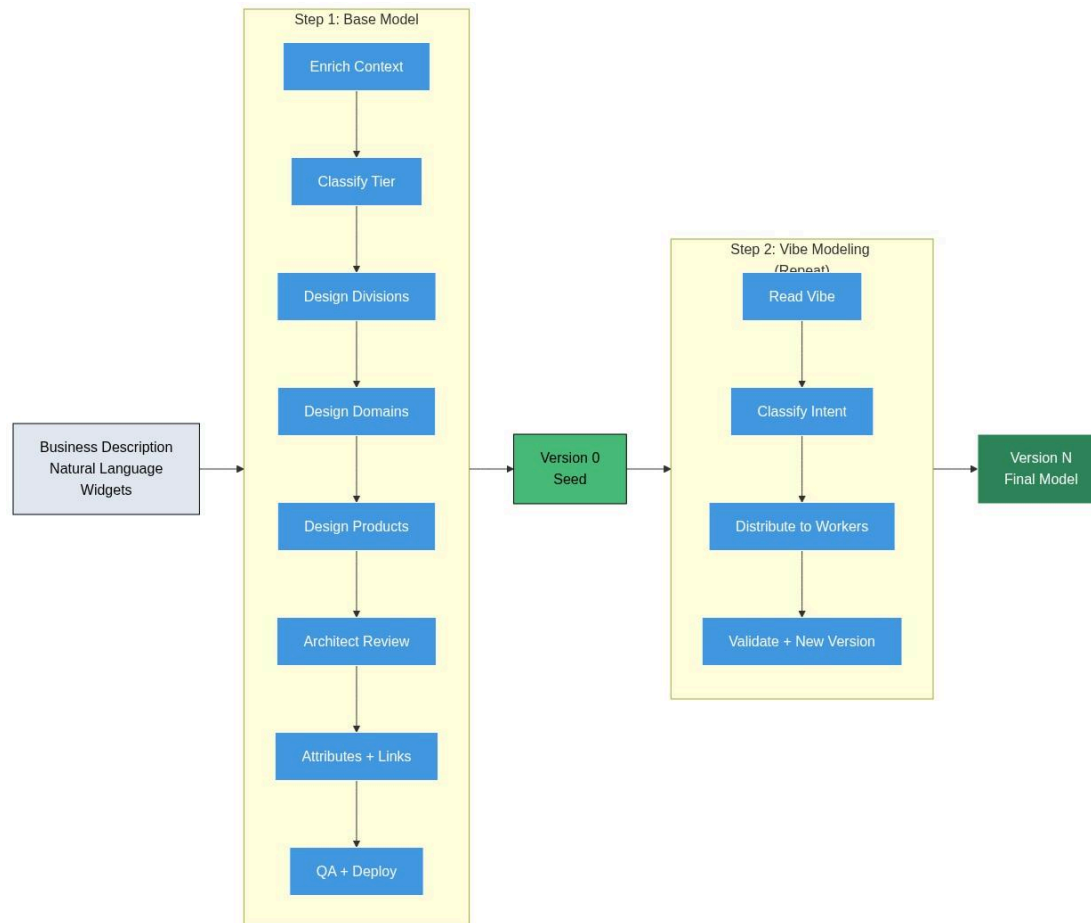
The rest of this paper is how a business data model gets built.

The Two Steps

Everything vibe modeling does fits into two steps. Understanding the boundary between them is more useful than memorizing any single feature.

Step one, the base model. The modeler describes their business. The agent generates a complete first cut by walking a single decomposition path: organization, divisions, domains, subdomains, products, attributes. Products are linked within their domain. Domains are linked across. The model is validated against the rule catalog. If a Unity Catalog was supplied, the schemas, tables, foreign keys, and tags are physically deployed. The output is version zero. A seed.

Step two, vibe modeling. The modeler reviews the seed. They write what they want changed in plain English. The agent classifies the intent, executes the change, and re-runs the same quality machinery. A new version is produced. No version is ever overwritten, so the work history is preserved.



Most of this paper is about step one, because it is where the scaffolding is built and where the rules have the most to do. Step two gets its own section later.

Step 1.1: Knowing the Business Before Modeling It

Every data model starts with a question. What does this business do.

The answer seems obvious to the people who work inside the business. It is not obvious to a system. The system needs structure. Not just the company name and a tagline, but a layered portrait of what this particular organization does, at several levels of detail.

The modeler provides that portrait through a set of widgets on the agent notebook. There are three groups.

Business information. The business name. A description of what the business does, written in plain prose. The industry alignment. The core business processes that happen every day. The organizational divisions the business uses. The data domains it already knows about. The jargon its people speak. The operational systems of record that currently hold its data. The governing bodies that regulate the industry.

For a telecoms operator, the jargon includes msisd, arpu, imsi, cdr. The governing body includes the GSMA and the national regulator. For a mining company the jargon is ROM, cut-off

grade, JORC. For a healthcare provider it is ICD, CPT, DRG, HIPAA. Every industry has its own language, and the model has to speak it.

Model conventions. These are the formatting rules for the artifact itself. Naming convention (snakecase by default, but camelCase, PascalCase, or SCREAMINGCASE are available). Primary key suffix. Primary key data type. Boolean representation. Date and timestamp formats. Classification levels. Prefix and suffix conventions for catalogs, schemas, and tags. Housekeeping and history-tracking column policy.

There are more than a dozen of these widgets, and they decide what every produced name will look like.

Vibe modeling instructions. A free-text field where, in step two, the modeler describes changes in natural language. For a new base model the field is empty.

Once those inputs are in, the agent does something a traditional tool cannot. It enriches the input.

The agent acts as a seasoned business analyst with deep industry knowledge. It expands every dimension.

If the modeler wrote "telecom" for industry alignment, the enrichment expands that into a precise characterization: mobile network operator, fixed-line broadband provider, converged services company. If the modeler listed three business processes, the enrichment expands them into ordered workflows. Billing becomes usage mediation, rating and charging, invoice generation, payment processing, dunning and collections, revenue assurance. Each expanded process implies specific data entities that the model will need.

This enrichment is not decoration. Every downstream stage reads it. A model that does not know what an msisdn is will generate useless column names in telecoms. A model that cannot distinguish a drill hole from a blast hole will fail in mining. The enriched context is the foundation. Alongside the enrichment, the agent classifies the business into one of five complexity tiers. This classification drives every sizing decision that follows.

Industry Complexity Tiers

The tier is based on seven dimensions, each scored zero or one.

1. Regulatory density. Three or more regulatory bodies imposing data and reporting requirements.
2. Party complexity. Three or more distinct party types (customers, suppliers, partners, employees).
3. Product hierarchy depth. Fifty or more product variants with complex bundling, pricing, lifecycle.
4. Infrastructure management. Owns physical or digital infrastructure requiring asset tracking.
5. Industry canonical model. Two hundred or more entity types defined by an industry standards body.
6. Transaction complexity. Ten or more distinct transaction types with multi-step lifecycles.
7. Operational system landscape. Five or more major systems of record across business functions.

If a business falls between two tiers the classifier rounds up. Better to slightly over-size than to under-serve. The figures below describe the full Expanded Coverage Model (ECM); a leaner Minimum Viable Model (MVM) is introduced later.

Tier	Label	Hallmarks	ECM Domains	ECM Products per Domain	ECM Attrs per Product	Subdomains per Domain
tier_1	Ultra-Complex	Banking, insurance, large pharma	15-22	14-28	15-50	3-6
tier_2	Complex	Telecoms, energy, healthcare	12-18	14-26	12-50	2-5
tier_3	Moderate	Manufacturing, retail	10-15	12-24	10-45	2-5
tier_4	Standard	Logistics, agriculture	8-12	10-20	10-40	2-4
tier_5	Simple	Consulting, SaaS, media	5-8	8-18	8-35	2-4

Everything that comes after this step, from the number of domains, to the number of products, to the attribute depth, reads the tier. The modeler can override any of these numbers through widgets if needed, but the tier gives the model a sensible default for an organization in that sector.

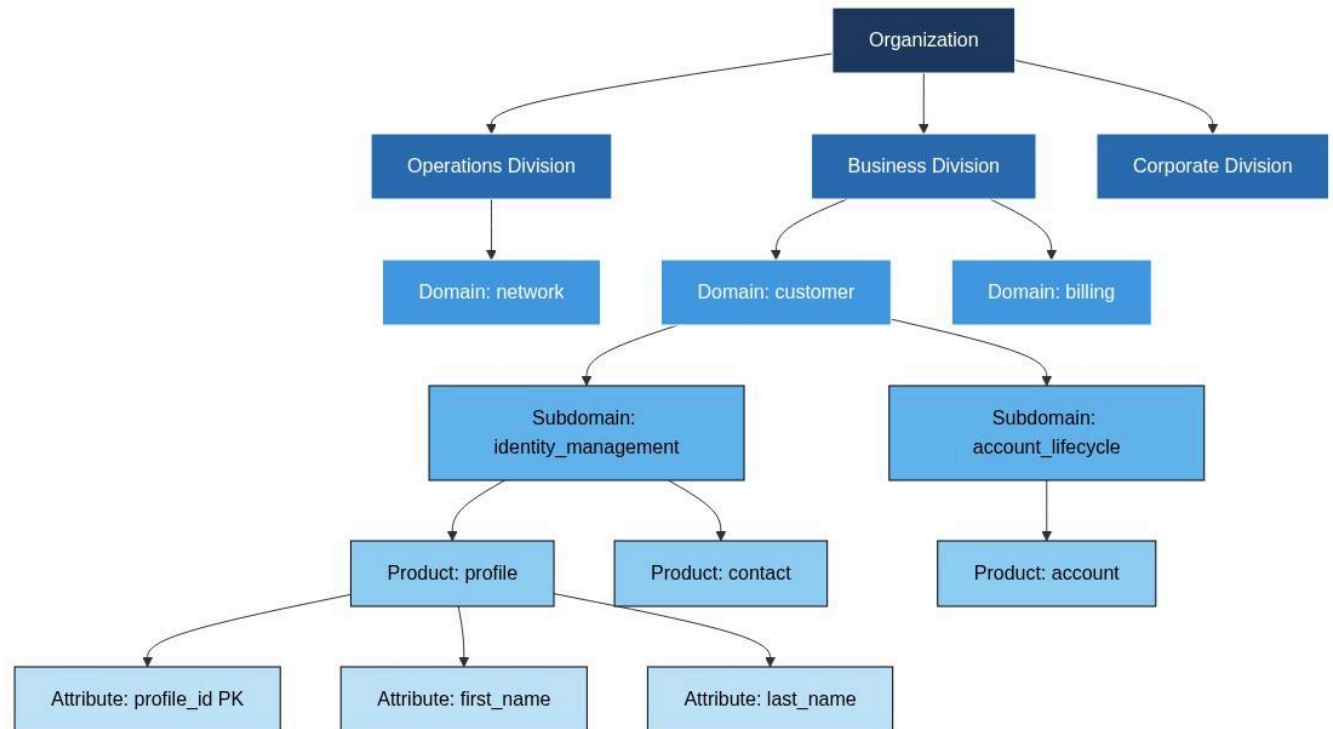
Before moving on, one thing has to be stated plainly. Nothing that follows is random.

Every step in the model's lifecycle is governed by a catalog of more than two hundred enforceable rules, organized into thirteen groups covering naming conventions, semantic deduplication, foreign keys, primary keys, normalization, domain structure, data types, tags, graph topology, quality, product design, sample data generation, and physical deployment. Each rule has an ID, a rationale, and an example. They are constraints, not suggestions. The modeler provides the soul. The rules provide the skeleton. The full catalog is published at `docs/whitepaper.md` and `docs/quality-gates.md` in the repository.

And every step follows the same disciplined cycle. Generate. Validate against the rules. If validation fails, feed the errors back and retry. Nothing moves forward until it passes scrutiny.

The Four-Level Hierarchy

Every model the agent produces follows the same strict hierarchy.



This hierarchy is not a suggestion. It is enforced by the rules, by the architect reviews, and by the static analysis that runs at the end of every pipeline.

Understanding it is the fastest way to understand any vibe model.

Step 1.2: Segmenting the Organization into Three Divisions

Before any domain is named, the agent establishes organizational scaffolding. Every business, regardless of industry, runs on three concentric rings of activity.

Operations is the innermost ring. What the business physically does to function. In telecoms: network management, service provisioning, fault management. In mining: mine operations, ore processing, fleet logistics. In healthcare: patient admissions, clinical procedures, pharmacy dispensing.

The test is direct. Could the business operate for one day without this division's data? If no, it is Operations.

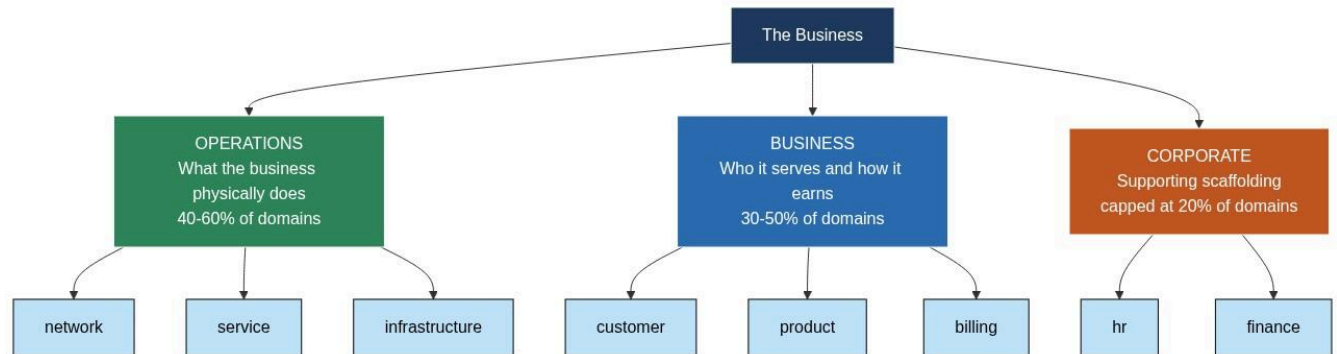
Business is the second ring. Who the business serves and how it earns revenue. Customer management, billing, product catalog, sales, contracts.

The test: does this area directly generate revenue or directly serve customers? If yes, it is Business.

Corporate is the outermost ring. The supporting scaffolding. Marketing, compliance, HR, corporate finance. These enable the business but do not directly generate revenue.

The test: could the business survive for a week without this division's data? If yes, it is almost certainly Corporate.

This three-ring structure is not a suggestion. It is rule G06-R001. It ensures the model is built from the inside out. Operations first, because they are the foundation. Business second, because revenue depends on operations. Corporate last, because it supports both. The division ratios are enforced. Operations plus Business must be at least eighty percent of all domains. Corporate is capped at twenty percent. No Corporate domain is considered until Operations and Business each have at least two domains of their own. The reason this matters. Unconstrained data modeling tends to produce models that are half HR, finance, and legal, and light on the operational core that actually runs the business. The three-ring constraint prevents that.



Step 1.3: Segmenting Divisions into Domains

A division is too broad to model directly. Nobody builds a database called Operations. The next step breaks each division into domains, which are the actual modeling units that will hold tables, columns, and relationships.

A domain is not a database. It is not a department. It is a bounded context. A coherent region within a division that owns a distinct set of concepts and speaks its own dialect. Think of each division as a continent and domains as the countries within it.

What is a Domain

A domain answers this question: within this division, what specific area of the business does this data belong to?

In a telecoms Operations division, network is a domain. It owns infrastructure, coverage, capacity. In the same company's Business division, billing is a domain. It owns invoices, charges, payments. Each domain owns a set of concepts that other domains may reference but do not own.

The customer domain owns who the customer is. The billing domain owns what the customer owes. The network domain owns what infrastructure exists.

The Org Chart Test

Every domain must pass a simple reality check (rule G06-R016): would a real department or team with this name exist in the organization?

Finance passes. Customer passes. Network passes for a telecoms operator. But utilities fails, because no one works in the Utilities department. Processing fails because it is too vague to be meaningful. If you cannot imagine a recruiter posting jobs in that department, it is not a real domain.

Generic catch-all names are forbidden (rule G06-R014). Services, platform, shared, common, core, misc, admin. These are buckets where unclear thinking goes to hide.

Analytics-oriented names are equally forbidden (rule G06-R015). Analytics, reporting, intelligence, metrics, kpi, datawarehouse. None of these are domains in a base model. The base model captures what happened, at the Silver layer of the medallion architecture. Derived analytics belong in the Gold layer above.

Naming Rules for Domains

Domain names have to be exactly one word, lowercase, singular, and no longer than twenty characters (rules G01-R002 and G01-R003). `customer` is valid. `customer_service` is not, because it is two words. `customers` is not, because it is plural. Exceptions exist for naturally singular terms that end in `s` like `logistics`, `sales`, `operations`.

Each domain gets a description of at least twenty characters explaining its business purpose. No two domains may have descriptions with more than seventy percent word overlap. If two descriptions sound the same, the domains are probably the same and should be merged.

How the Domains Are Chosen

The agent does not rely on a single judgment. It solicits four independent domain proposals, each generated with a different reasoning style.

Then a fifth, senior judge reviews all four side by side. The judge looks for consensus, unique insights, and redundancy. If one proposal calls the domain `customer` and another calls it `subscriber`, the judge recognizes these as the same concept and picks one canonical name, enforcing rule G06-R006 (each business concept is owned by exactly one domain).

The order of selection follows the division hierarchy. Operations first. Business second. Corporate last. The interleaving algorithm (rule G06-R005) alternates between Operations and Business so neither is starved at the expense of the other.

The outcome is a set of domains, each classified into one of the three divisions. The continents are divided into countries. The borders are set.

Step 1.4: Filling Domains with Tables

With the domains established, the model enters its most productive phase. Each domain is filled with products, which are the tables that hold the business's data.

All domains are populated in parallel. For each domain, the agent generates the tables that naturally belong there.

In `customer`: profile, account, address, contact, preference, segment. In `billing`: invoice, payment, charge, credit_note, dispute. In `network`: element, alarm, incident, coverage, site.

Business Concepts Only

Every table must represent a genuine business concept that a domain expert would recognize. Technical artifacts such as `etl_job_log` or `batch_control` are plumbing, and they belong in operational schemas, not the business model (rule G12-R001). Analytics tables like `churn_prediction` or `revenue_analysis` are derived insights that belong in the Gold layer (rule G12-R002). Speculative content like `blockchain_hash` or `quantum_encryption_key` represents what the business might do someday, not what it does today (rule G12-R004). All three are turned away.

Single Source of Truth

After every domain has its tables, the agent runs a global semantic deduplication pass. It scans every table across every domain, looking for duplicates. Tables that represent the same business concept under different names. `invoice` and `bill`. `customer` and `subscriber`. The SSOT principle (rule G02-R001) demands that each concept has exactly one authoritative home.

Deduplication is conservative. Not everything that looks similar is the same.

- Different lifecycle stages are not duplicates (rule G02-R006). A `quote` and an `order` represent different stages of the same commercial process. Both should exist.
- Different granularity levels are not duplicates (rule G02-R007). Individual usage events and monthly summaries serve different purposes. Both stay.
- Parent and child entities are not duplicates (rule G02-R008). `order` groups items. `order_item` details them. Both stay.

When genuine duplicates are found, merging is preferred over deletion (rule G02-R009). A discriminator column is added to the merged table to preserve which domain the original record came from.

Table Classification

Tables are classified by nature (rule G07-R008).

Master data (customer, product) describes what entities are. Authoritative records that change slowly. Reference data (country, currency) provides static lookups. Transactional data (order, payment) captures what happened. Association data (enrollment) represents many-to-many relationships.

Every table is also classified as core or helper (rule G12-R007). Core tables are essential, directly queried by stakeholders. Each domain should have one to three core tables maximum (rule G12-R009). If everything is core, nothing is.

Naming Rules for Products

Table naming is strict. One to three words, lowercase, under thirty characters (rule G01-R004).

Names never repeat the domain name as a prefix (rule G01-R005). In the customer domain the table is `account`, not `customer_account`, because the full path `customer.account` already provides context.

Each table gets a primary key following a sacred convention (rules G04-R001 and G01-R007). The table name followed by `_id`. The `customer` table gets `customer_id`. The `order_item` table gets `order_item_id`. When you see `customer_id` in any table, in any domain, you immediately know it references the customer table. The model becomes self-documenting.

Step 1.5: The Architect Reviews

Before attributes are generated, the model is subjected to two levels of architect review. This is one of the most distinctive parts of the pipeline and deserves its own section.

Domain Architect Review (per-domain, in parallel)

A first pass of tables has been drafted inside every domain, but drafted is not reviewed. Each domain is handed to a reviewer that holds two voices at once.

The Principal Data Architect voice knows the rules of sound modeling. The Senior Business Subject Matter Expert voice knows how a real practitioner in that specific domain and industry thinks.

All domains are reviewed in parallel. One independent reasoning pass per domain, bounded by the same concurrency limit the rest of the model uses. This is deliberate. Each reviewer focuses only on their assigned domain and nothing else. Cross-domain concerns are explicitly out of scope and belong to the next step. If a domain is too large for a single LLM context window, the reviewer batches the content internally rather than truncating.

Each reviewer produces two kinds of output.

The first kind is actionable and applied immediately. Products to add, products to rename, products to remove. Description improvements. In-domain foreign key links, which are queued rather than created directly so the downstream linker can honor them with full rule enforcement. The second kind is deferred. Merges or splits that need attribute-aware reconciliation. Blockers or required actions surfaced by failed production-readiness gates. These are stashed alongside the model so the next vibe iteration can address them.

Every reviewer runs the same four production-readiness gates, scoped to their domain:

1. Would I trust this domain in production?
2. Would I support it in production?
3. Would I recommend it to my industry peers?
4. Would I propose it for a global industry standard?

Each gate returns Yes or No, with explicit blockers and required actions. A single No does not fail the pipeline. It feeds the `next_vibes.txt` output so the modeler can decide whether to act on it in the next iteration. Some of these questions are intentionally a stretch, for example, propose it for a global industry standard, the main reason behind it is to push the model quality further and further.

The prompt that drives this review is industry-agnostic. It speaks only in placeholders such as {domain_name}, {industry_alignment}, {business}, so the same logic works for any organization in any sector the user describes.

Global Architect Review (cross-domain, holistic)

With every domain now internally reviewed, a single Global Architect reviews the model as one organism. Its scope is everything the per-domain reviewers deliberately could not touch.

Cross-domain SSOT. The overall shape and balance of the domain set. Essential cross-domain foreign keys. Higher-level structural changes like adding, removing, or renaming a domain, or moving a product from one domain to another.

The division of labor is strict. The per-domain reviewer may add, rename, remove, merge, or split products within one domain. The global reviewer owns everything that crosses domain boundaries.

This prevents a common failure mode, in which a global reviewer over-optimizes the shape of the model and accidentally loses a product that only made sense to the domain's own expert, or in which a domain reviewer invents a cross-cutting structural change that contradicts another domain.

The global reviewer runs the same four production-readiness gates, now scoped to the whole model. Failures at either level, per-domain or global, flow into the same stash of blockers and required actions.

The two reviews are complementary, not redundant. The per-domain reviewer catches what only a practitioner inside one bounded context can see. The global reviewer catches what only an observer of the whole can see.

Fifteen Scorecards

Beyond the four gates, each review produces a fifteen-dimension scorecard covering domain coherence, SSOT integrity, granularity consistency, FK direction correctness, naming quality, description quality, core/helper designation appropriateness, attribute depth readiness, business-term usage, duplication avoidance, industry alignment, cross-domain relationship candidates, production readiness, governance readiness, and regulatory readiness.

The scorecard is what produces the confidence score that ships with every model version.



Step 1.6: Filling Tables with Attributes

Every table now has a name, a classification, and a primary key. But a table without columns is just a label. This step gives each table its substance.

Attribute generation runs for every table in parallel. The agent knows what each table type demands (rule G12-R018).

A person entity without name and contact fields is a ghost. An account without type, status, and key dates is an empty shell. A transaction without a date or an amount is a record of nothing.

No Trivial Attributes

Every column must earn its place (rule G12-R017). The litmus test: what specific business question does this column answer?

If the column `subscription_start_date` answers "When did this subscription begin?" it stays. If `notes_field_3` answers nothing specific, it goes. No filler. No padding. No speculative columns.

Pre-calculated metrics like `average_monthly_spend` or `churn_score` are refused (rule G12-R003). These are derived insights. The base model stores raw ingredients like individual

charges and individual usage records, and lets the analytics layer compute the totals and scores.

Naming Rules for Attributes

Column names never repeat the table name as a prefix, except for the primary key (rule G01-R006). In `account`, the column is `balance`, not `account_balance`.

Column names prioritize clarity over brevity (rule G01-R013). Always `mean_time_between_failures`, never `mtbf`. Industry jargon earns a place when universally recognized (rule G01-R015), so `msisdn` in telecoms, `jorc_code` in mining, `iban` in banking. But lazy abbreviations like `cust_nm` for `customer_name` never do.

Data types are inferred from naming patterns (rule G07-R011). `_id` columns get integers. `_date` gets DATE. `_at` gets TIMESTAMP. Monetary suffixes get DECIMAL. No complex types like ARRAY, STRUCT, or MAP are allowed (rule G07-R002). Nested data is modeled as separate, normalized tables.

Foreign key columns must end with the target table's primary key name (rule G01-R008). Descriptive prefixes are allowed. `billing_address_id` and `shipping_address_id` both validly reference `address.address_id`.

Column Ordering

Within each table, columns follow a strict seating arrangement (rule G12-R021). Primary key first. Foreign keys second. Business attributes third. Housekeeping columns fourth. History-tracking columns last.

This ordering makes every table scannable and predictable across the entire model.

Step 1.7: In-Domain Linking

Tables exist. Columns exist. But without relationships they are a collection of isolated spreadsheets.

The web of foreign key relationships is what transforms tables into a model.

In-domain linking is where the web begins. It starts inside each domain because the relationships there are the most natural and the most obvious. Tables that belong to the same business area are almost certainly connected.

Take the `customer` domain with tables for profile, account, address, preference, segment. An account belongs to a profile. An address belongs to a profile. A preference belongs to a profile. These are not arbitrary connections. They are the real-world truth that a customer's account, address, and preferences all belong to that customer.

Why In-Domain First

In the real organization, the people who work in billing mostly work with other billing tables. The people who manage the customer domain mostly join customer tables to other customer tables.

In-domain relationships are the high-frequency joins, the ones that analysts run hundreds of times a day. Getting them right is the foundation that cross-domain linking will build on.

The Foreign-Key Rules

Every foreign key must point to the actual primary key of its target (rule G03-R001). Data types must match (rule G03-R004). Foreign keys always follow the child-to-parent direction (rule G09-R005).

`order.customer_id` is correct. `customer.latest_order_id` is wrong because it inverts the relationship. Bidirectional keys are forbidden (rule G03-R003), because they create cycles. Self-references are blocked unless hierarchical (rule G03-R002).

`category.parent_category_id` is legitimate. A random self-reference is not.

When a foreign key is established, normalization follows automatically (rule G03-R010). If

`order` gains `customer_id`, then redundant columns like `customer_name` and `customer_email` are removed, because those values are now accessible through the join.

The order is always the same. Add the key first. Remove the redundancy second.

Before linking even begins, a normalization integrity check runs on every table in parallel. It finds orphaned foreign key columns (columns ending in `_id` with no formal link declared) and either connects them to their obvious targets or flags them for resolution. It also finds denormalization violations (columns that copy data from another table, rule G05-R001).

Redundant columns are removed. The join path replaces them.

After the domain-level linking pass, a batch semantic resolution step sweeps the entire model, collecting every remaining unlinked `_id` column and resolving it in one pass.

Step 1.8: Cross-Domain Linking and the DAG

If in-domain linking connects the rooms of a house, cross-domain linking connects the houses into a city.

This is where the model becomes a true enterprise data mesh.

The organization is one body. The billing department serves the customer department's customers. The network department's infrastructure carries the product department's offerings. The service department resolves issues that originate in the network domain.

These connections must exist in the data model, or cross-domain analytics, the entire reason an enterprise model exists, becomes impossible.

The agent examines every unique pair of domains. For a model with seven domains, that is twenty-one pairs. For twenty-one domains, that is two hundred and ten pairs. Each pair is examined in parallel. How does billing connect to customer. How does service connect to network. How does logistics connect to procurement.

No domain may be an island (rule G09-R004). Every domain must connect to at least one other through foreign key relationships. An isolated domain is a data silo that cannot participate in cross-domain analytics. An analyst asking "which products generate the most billing disputes?" needs to traverse from product through billing to get the answer. If those domains are not connected, the question is unanswerable.

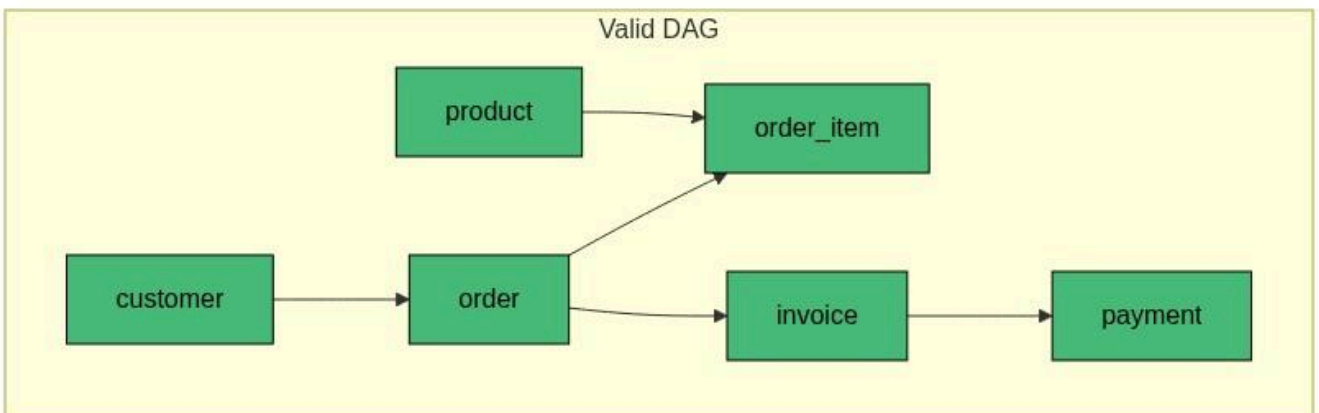
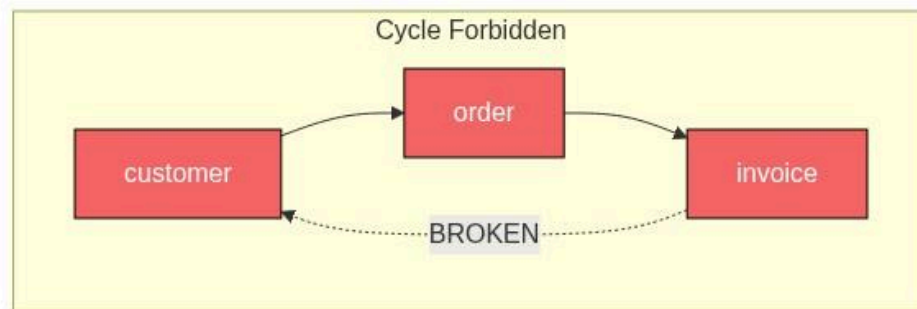
Before creating any cross-domain link, the agent performs an identity check on each domain pair. Are these two domains actually the same domain accidentally split? If they share five or more products with the same names and their functional overlap exceeds seventy percent, they are flagged as identical and recommended for merger.

Cross-domain many-to-many relationships are treated with extreme conservatism (rule G12-R013). The agent expects at most zero to one per model. The tests are stringent. Is the relationship genuinely bidirectional? Does the business track at least two attributes about the relationship itself? Does the business have a name for this relationship?

"Customers subscribe to products with terms, dates, and tiers" is a genuine many-to-many. "An order belongs to one customer" is a simple foreign key.

The DAG

Throughout this phase, the model enforces its most important structural law (rule G09-R001). The data model must form a Directed Acyclic Graph.



Follow foreign key relationships from any table and you must never return to the starting table. Cycles make it impossible to determine insertion order, break referential integrity, and create infinite loops.

When cycles are detected, the agent breaks the weakest link. Convenience references like `latest_order_id` or `primary_invoice_id` are always broken first (rule G09-R007). Core structural relationships like `order_item.order_id` are never broken (rule G09-R008).

Step 1.9: Subdomain Allocation

Domains alone are often too coarse for a human reader. A `customer` domain in a `tier_1` ultra-complex business might hold twenty tables. A human looking at that list wants structure inside that structure.

The agent allocates every domain's products into subdomains. Semantic groupings of related tables that sit between the domain and its products.

The rules (SUB group):

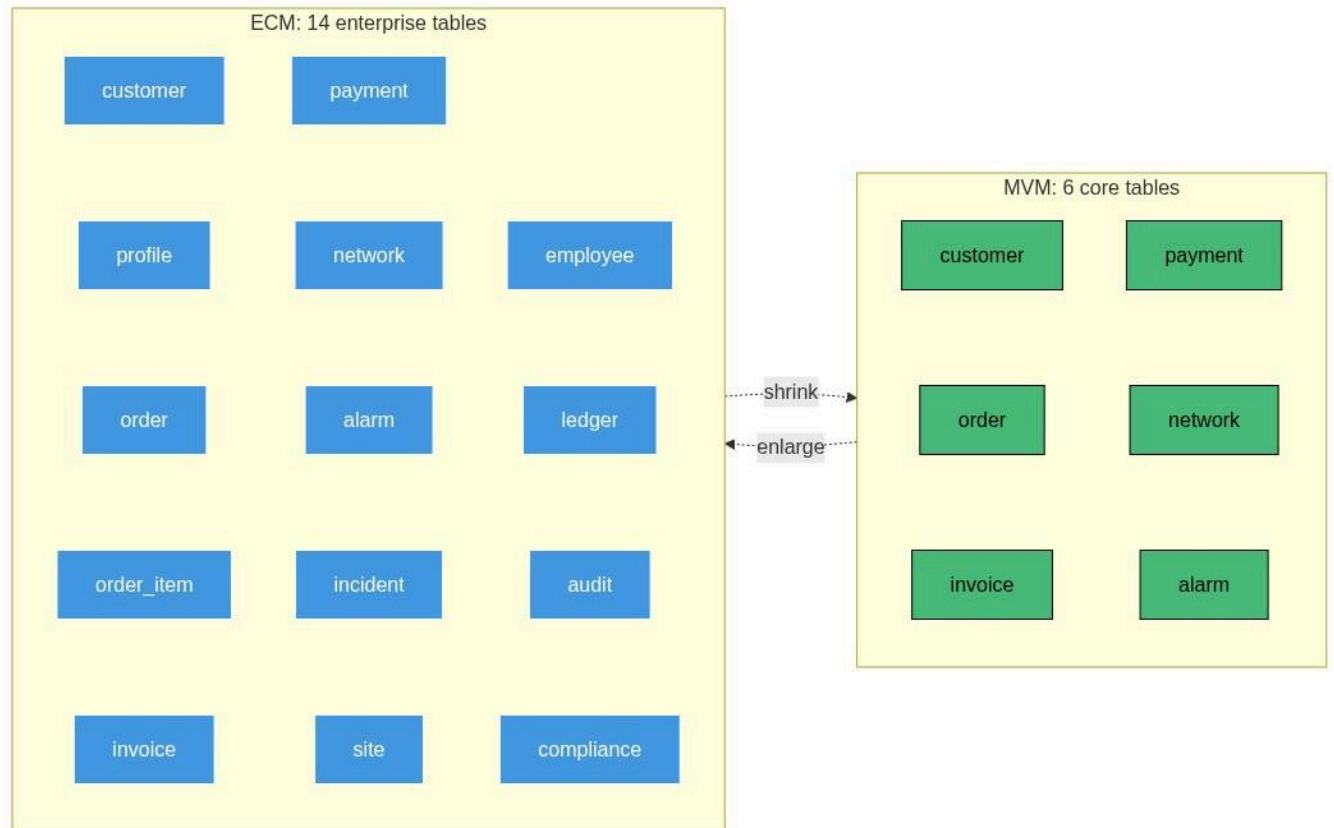
- **SUB-R001.** Min and max subdomains per domain come from the complexity tier. Never exactly one subdomain per domain. A domain has zero or two or more subdomains.
- **SUB-R002.** Subdomain names are exactly two words. One word is rejected. Three or more words are rejected. The two-word constraint forces thoughtful grouping and produces a readable label like `identity_management`, `kyc_verification`, `anode_production`.
- **SUB-R003.** Every subdomain must contain at least the minimum products per subdomain defined by the tier. ECM *tier1 through tier4* requires at least three. ECM *tier_5* requires two. MVM tiers are slightly looser.
- **SUB-R004.** No two subdomains within the same domain may share any word in their names. This prevents `customer_profile` and `customer_identity` in the same domain, because they both lead with the same word.
- **SUB-R005.** Subdomain names must use business terminology, not technical terms. `account_lifecycle` is fine. `table_grouping` is not.
- **SUB-R006.** Products are distributed as evenly as possible across subdomains within a domain.
- **SUB-R007.** Placeholder names are forbidden. Anything that looks like `Sub Domain1`, `Category 1`, `Group A`, `N/A`, `Other`, `General`, `Miscellaneous`, `Uncategorized` is rejected.
- **SUB-R008.** Each subdomain belongs to exactly one parent domain. No subdomain drift across domain boundaries.

Example. In the `party` domain of a *tier1 bank*, *subdomains might be* `'identitymanagement'`, `kycverification'`, `'partyengagement'`, `consent_lifecycle'`. Each one holds three or more products. Each one is a clean phrase a business person would recognize.

Subdomains are mostly an organization aid for humans. They also matter physically, because they become schema organization hints when you pick the right cataloging style.

Step 1.10: Model Scopes, MVM vs ECM

Two scopes are available and the choice drives the entire sizing of the model.



Minimum Viable Model (MVM). Thirty to fifty percent of the ECM table count. Essential business functions only. Ideal for SMBs, rapid deployments, proofs of concept, dev and test environments.

And this is the important part: attribute depth is the same as ECM. MVM is not a skeleton or a demo toy. Every table in an MVM has the same attribute min and max bounds as the same tier's ECM. The lightness comes from fewer domains and fewer tables, never from thinner tables.

Expanded Coverage Model (ECM). Full coverage. All divisions including Corporate back office. All the domains a Fortune 100 model would expect. Maximum breadth.

Why both scopes matter. A small business that starts with an ECM will spend months deleting. A Fortune 100 enterprise that starts with an MVM will spend months adding. Starting with the right scope is itself time saved.

Moving Between Scopes

Two operations are available for scope changes.

shrink `ecm` takes an ECM version and produces an MVM subset in a new version. It is a real LLM-guided transformation, not a random sample. It protects core products, keeps essential FKs, enforces the no-silo invariant, and emits a `next_vibes.txt` that explains what it kept and why.

enlarge `mvm` does the opposite. It takes an MVM version and expands into a full ECM in a new version. It uses the same widget-driven context so the added products respect the naming conventions and the governance choices already made.

No version is ever overwritten. Shrink or enlarge, both create a new numbered version alongside the original.

Step 1.11: Quality Checks, Naming, and Physical Deployment

Everything built so far (domains, tables, columns, relationships, subdomains) was constructed in stages, each following its own rules.

But has the whole held together?

A city built neighborhood by neighborhood can still have mismatched roads, dead ends, and buildings that violate the zoning code. This step holds the entire model up to the light.

The QA Suite

It begins at the edges. Empty domains are removed (rule G11-R023). Small tables are evaluated for merging (rule G12-R026). Every table is verified to have a primary key (rule G04-R001).

Then it turns inward to the structural backbone. The agent identifies its core products and marks them as protected. A fresh global semantic deduplication scan runs, because linking and normalization may have revealed new overlaps.

Global product-name conflicts across domains are investigated. `customer.device` (a phone) and `iot.device` (a sensor) share a name but represent different concepts, so both are kept. `billing.payment_record` and `finance.payment` with ninety percent overlap are genuine duplicates, and one is consolidated.

The most complex check is graph topology. Depth-first cycle detection traverses the entire relationship web. Any remaining cycles are broken following the established hierarchy.

Convenience references first. Structural relationships never. No break may create a disconnected table. Siloed tables (tables with no incoming or outgoing relationships) are detected (rule G09-R003) and either reconnected or removed.

The audit then zooms out to architecture. Product-domain location fit is examined. Are tables in the right domain? Tables cannot cross division boundaries (rule G06-R008). Eponymous tables like `mine` in the `mining` domain can never be relocated (rule G06-R010). Child tables stay with their parent (rule G06-R011). Domain balance is validated. Operations and business domains must make up at least eighty percent of all domains.

Semantic distinction rules prevent false-positive duplicates (G02-SD group). `payment_method` and `payment_channel` are not the same thing (SD01). `sla_target_time` and `sla_actual_time` answer different questions (SD03). `created_at` and `shipped_at` mark different milestones (SD04). A model that loses real business nuance in the name of tidiness is worse than one that tolerates a few extra columns.

Naming Convention Application

After the structural QA passes, naming conventions are applied across the model in a single deterministic pass.

The naming convention widget picks one of four cases (*snakecase by default, camelCase, PascalCase, SCREAMINGSNAKE_CASE*). Every domain, schema, table, and column is rewritten to that case. Prefixes and suffixes from the model-convention widgets are applied. Catalogs pick up their configured prefix and suffix. Tables preserve history-tracking columns if that option is on. Classification tags pick up their configured naming. Because this happens in one pass, the resulting model is completely consistent. No half-renamed tables. No columns in one case and tables in another.

The Logical-Physical Separation

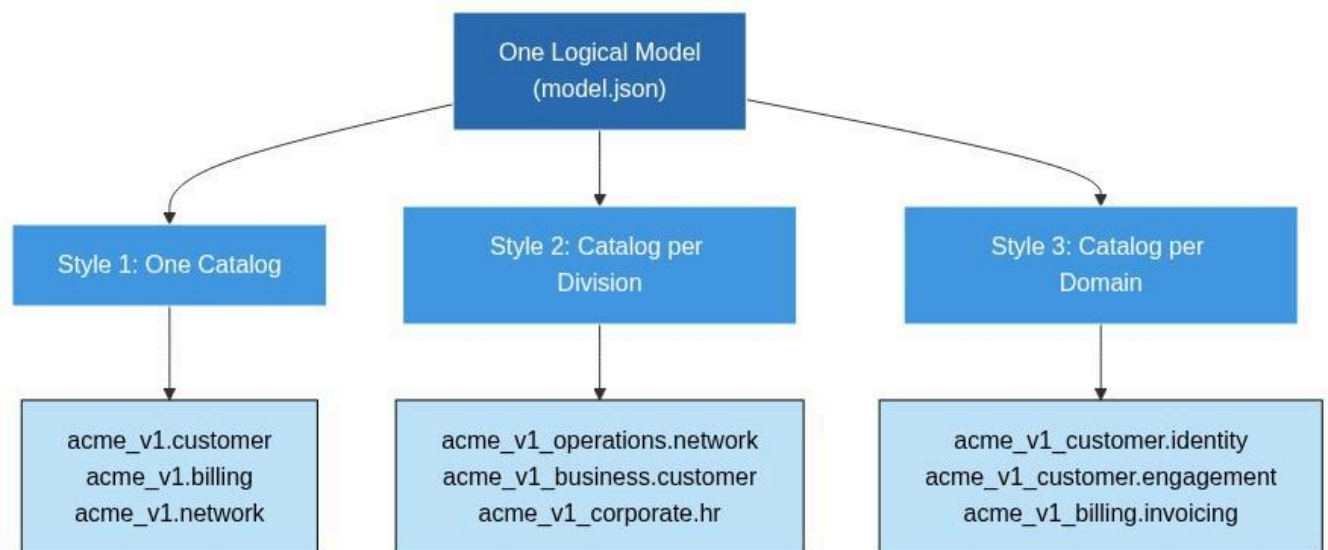
This is the feature that makes Vibe Data Modeling portable, reusable, and safe to redeploy across any environment your organization can dream up.

The agent produces **one logical model**, stored as a single `model.json` file.

This file captures every domain, subdomain, product, attribute, relationship, classification tag, and metric-view definition. It is a complete, self-contained description of what the business is. It does not know where it will be deployed. It does not know which catalog name to use. It does not care.

The `model.json` file is the unit of currency in Vibe Data Modeling. You can check it into git. You can diff two versions. You can share it across environments. You can hand it to a security reviewer without giving away production access.

Alongside `model.json`, the agent also offers **three physical layouts** in Unity Catalog, called **cataloging styles**. The same logical model deploys differently depending on which style you pick.



With **One Catalog**, the entire business goes into a single Unity Catalog catalog, and each domain becomes a schema inside it. A telecom might have `acme_v1.customer`, `acme_v1.billing`, `acme_v1.network`.

This is best for organizations that want one governance boundary, one set of tags, one set of lineage trails. It keeps cross-domain joins fully qualified with a short path.

With **Catalog per Division**, each division (Operations, Business, Corporate) gets its own catalog, and domains become schemas inside their division catalog.

That same telecom would have `acme_v1_operations.network`,
`acme_v1_business.customer`, `acme_v1_corporate.hr`.

This is best for large enterprises that govern divisions separately. The HR catalog and the Operations catalog can have different admins, different retention policies, different classification regimes. Cross-division queries are still straightforward.

With **Catalog per Domain**, each domain gets a dedicated catalog, and subdomains become schemas inside it.

The telecom would have `acme_v1_customer.identity_management`,
`acme_v1_customer.engagement`, `acme_v1_billing.invoicing`,
`acme_v1_billing.collections`.

This is best for federated organizations where each domain is its own product with its own owner. It maps cleanly onto a data-mesh operating model. Each domain catalog can be shared, tagged, and governed independently.

All three styles deploy from the same `model.json`. All three preserve the same logical relationships, the same classifications, the same metric views. If your governance reality changes, redeploy to a different style and nothing about the logical model is touched.

This is also why round-tripping is possible. You can take a `model.json` produced in dev, redeploy it to staging under a different cataloging style, redeploy it again to prod under a third style, and get three environments whose logical content is byte-identical. There is no drift, because the logical truth and the physical layout are intentionally kept in separate layers.

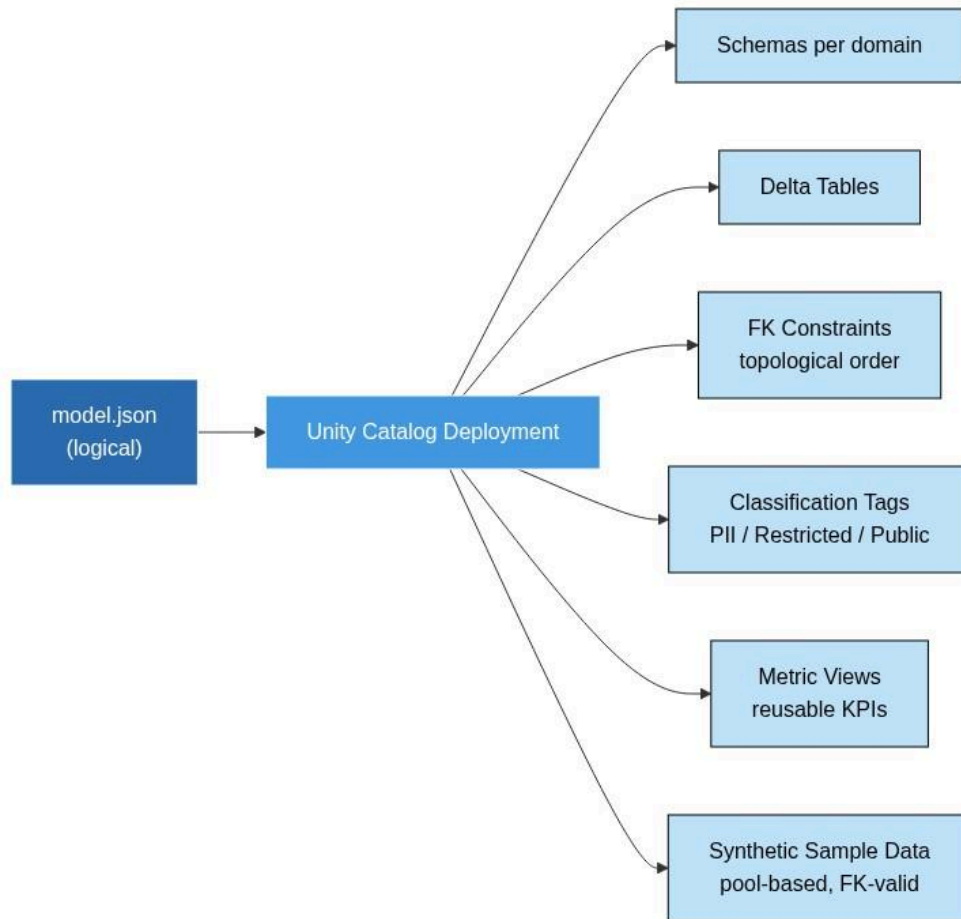
Step 1.12: Physical Deployment

After the cataloging style is chosen, the physical deployment pipeline runs.

Schemas are created in parallel, one per domain (or one per subdomain, depending on the style). Delta tables are created in parallel. Foreign key constraints are applied sequentially, in dependency order, using Kahn's algorithm on the DAG to ensure every parent exists before its children.

Data classification tags are applied to every column that needs them. Metric views are installed on top of the physical tables as reusable KPI definitions. If sample-data generation was requested, synthetic rows are written to every table, with foreign keys guaranteed to resolve. At the end, a static analysis examines the entire deployed model and produces a confidence score plus a set of forward-looking recommendations. Things like "the billing domain has only four tables, consider adding `chargedetailrecord`" or "the network domain has no connection to customer, consider adding `service_id` on the incident table."

Version zero, the seed, is now complete. Deployed. Tagged. Classified. Self-aware.



Those recommendations are the bridge to step two. The seed is planted. Now the modeler tends it.

Step 2: Vibe Modeling

The seed model is version zero. A solid, rule-compliant, fully deployed starting point. But it is still a starting point. It was built by a system that understands the business well but has not yet heard from every stakeholder.

The billing team has opinions. The network operations team has requirements. The data governance committee has naming standards. None of them were in the room when version zero was born.

Vibe modeling is how their feedback reaches the model. The modeler writes natural-language instructions, plain English, not SQL, not JSON, not entity-relationship diagrams, and the system translates those words into precise, validated changes.

Each vibe creates a new version (v1, v2, v3), each one iterating on the seed, moving it closer to the ideal. No version is overwritten. The model evolves.



Understanding Intent: Surgical, Holistic, Generative

Before executing anything, the agent must understand what the modeler wants. It reads the entire instruction in context and performs a five-dimensional analysis of intent.

Preservation (keep the model intact or rebuild?). Scope (specific entities or broad principle?). Change philosophy (targeted fix or new content?). Satisfaction (polishing or frustrated?). And the core desired outcome.

That analysis picks one of three modes.

Surgical mode. The modeler has identified specific problems with specific entities. Implicit contract: "Fix these things. Leave everything else alone."

"Remove the duplicate status column from the invoice table."

"Fix the FK from order to warehouse."

"Drop the marketing domain."

In surgical mode, the agent modifies exactly what was asked. No content is regenerated. No tables are rewritten. If the modeler asked to remove three columns and rename one, exactly four changes occur.

Holistic mode. The modeler wants a rule or pattern applied consistently across the entire model, without changing its structure. Implicit contract: "Apply this principle everywhere, but preserve the structure."

"Ensure all primary keys follow the naming convention."

"Add *createdat* and *updatedat* to every table."

"Make sure every *_id* column is linked."

In holistic mode, the agent touches many tables, potentially all of them, but formulaically. No creative decisions. No new tables unless explicitly requested. Just consistent application of a rule across the entire model.

Generative mode. The modeler wants something new. Implicit contract: "Create or rebuild this part to be better."

"Add a loyalty domain with program, tier, reward, and enrollment."

"This model feels too generic for telecom. Make it more industry-specific."

"Split billing into invoicing and collections."

In generative mode, the modeler's instructions are decomposed into twelve tailored briefings, one for each downstream worker (domain generation, product generation, attribute generation, in-domain linking, cross-domain linking, many-to-many detection, semantic deduplication, global deduplication, product merge, attribute deduplication, normalization, cycle breaking).

Each worker receives not a generic copy of the vibe but a custom briefing with specific requirements, enriched guidance, explicit do's and don'ts, cautions about potential mistakes, and quality criteria. The result is creative without being chaotic. Twelve workers, each interpreting the same vision through their own expertise.

After the Change

After any change, surgical, holistic, or generative, the model re-enters the same quality pipeline from step one.

The same rules apply. The same checks run. A table born during a vibe receives the same scrutiny as a table born during the original creation. Vibe modeling does not shortcut quality.

Auto-Generated Next Vibes

After every run (base model or vibe iteration), the agent produces a `next_vibes.txt` that lists recommended next-step vibes based on the static analysis.

You can copy it directly into the Vibe Modeling Instructions widget for the next version and iterate.

Example Progression

Follow a telecoms model through four versions.

Version zero. The seed from step one. Seven domains, forty-eight tables, five hundred columns, ninety-six foreign keys. Static analysis finds fifteen issues. Confidence score 72. Recommendations: fix three broken FKs in billing, link eight unlinked `_id` columns, connect the isolated workforce and procurement domains, consider splitting the oversized product table.

Version one. The modeler takes the recommendations and adds a twist. "Fix the broken FKs. Link all unlinked *id* columns. *If the parent table doesn't exist, create it. Connect workforce to service. Split product into catalog and pricing. Use telecom jargon everywhere: msisdn not mobilenummer, imsi not subscriber_identity.*"

The agent classifies this as generative, distributes the vibes to all twelve workers, and produces a refined model. Six issues remain. Confidence 85.

Version two. The second vibe. "Merge *servicerequest* and *workorder* into *serviceorder* with a discriminator. Connect procurement to product through *supplierproduct*. Add a loyalty domain. Don't touch billing or network."

Generative mode with explicit protection. The merge executes, loyalty is created and populated, procurement connects to product. Two minor issues remain. Confidence 93.

Version three. The third vibe. "Link *loyalty.enrollment* to *customer.profile*. Fix the naming in workforce."

Surgical. Two targeted actions. Zero issues. Confidence 97.

Four versions. Minutes, not months. Each built on the seed. Each guided by the model's own self-assessment. And because every version is preserved, the modeler can compare any two, roll back if needed, or branch the line.

The DAG-First and Ontology-First Mindset

There is a design philosophy that runs beneath every step described above. It deserves to be stated explicitly.

The model is not built as a flat collection of tables. It is built as a graph, specifically a Directed Acyclic Graph, and as an ontology. This dual nature is what makes the output useful far beyond traditional SQL analytics.

DAG-First

Every foreign key relationship has a direction. From child to parent. `order.customer_id` points from order (child) to customer (parent).

This directionality is a semantic statement about how the business works. An order belongs to a customer. A customer does not belong to an order.

By enforcing the DAG constraint (rule G09-R001) at every step, the model guarantees three production properties.

1. **Insertion order is always deterministic.** Parent tables are loaded before children. Foreign key constraints can be applied without conflict. ETL pipelines can be built by following the graph from roots to leaves.
2. **Traversal always terminates.** Any query that follows foreign key paths will terminate. Any recursive traversal will converge. Any lineage trace will end at a root table.
3. **The model is topologically sortable.** Foreign key statements are sorted in dependency order using Kahn's algorithm before physical deployment. This makes schema creation deterministic and repeatable.

Ontology-First

Alongside the physical schema, the agent generates an RDFS (Resource Description Framework Schema) representation.

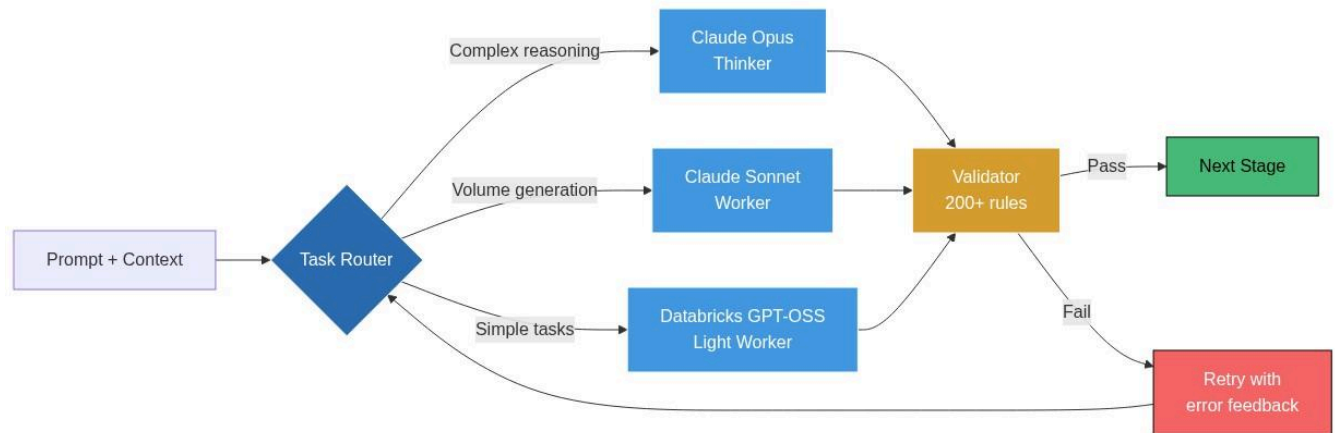
Every domain becomes a class. Every table becomes a subclass. Every attribute becomes a property with a defined range. Every foreign key becomes a semantic relationship. The result is both a database schema and a knowledge graph.

Same model. Multiple lenses. SQL analysts query the physical tables in Unity Catalog. Data scientists ingest the machine-readable `model.json`. Knowledge graph platforms consume the RDFS ontology. Governance tools read the tags. Lineage tools traverse the DAG.

The ontology also makes the model discoverable. Semantic search can find tables by meaning, not just by name. "Show me everything related to customer payments" can traverse the ontology from the customer class through the billing relationship to the payment class, even if the analyst does not know the exact table names.

The LLM Architecture Under the Hood

The agent is not one model. It is a multi-model ensemble with automatic demotion and recovery.



Role	Purpose	Endpoint
Thinker (large)	Complex reasoning. Architecture reviews. QA decisions.	`databricks-claude-opus-4-6`
Worker (large)	High-volume generation. Products, attributes, FKs, dedup.	`databricks-claude-sonnet-4-6`
Thinker (fallback)	When the primary thinker is saturated.	`databricks-claude-opus-4-5`
Worker (fallback)	When the primary worker is saturated.	`databricks-claude-sonnet-4-5`
Worker (small)	Simpler tasks. Domain generation, tag classification.	`databricks-gpt-oss-120b`
Worker (tiny)	Sample data generation.	`databricks-gpt-oss-20b`

After three cumulative failures on a model, the entire priority order is demoted, the failing model is pushed down, and the next model takes its place. After five consecutive successes, the original order is restored. After three consecutive timeouts, a model is marked broken and skipped for the rest of the session.

Forty-nine specialized prompts run in the agent today. Each is mapped to a specific model role and temperature. Thinker prompts use temperature 0 for deterministic reasoning. Worker prompts use temperature 0 to 0.3 for controlled generation. Sample data generation uses temperature 0.5 for creative variety within a constrained pool-based engine.

Every step in the pipeline follows the same four-beat rhythm. Generate. Validate. If validation fails, feed the errors back and retry. Accept or discard.

Nothing moves forward until it passes scrutiny.

Metric Views

Databricks metric views sit on top of your physical tables. They are reusable KPI definitions with dimensions (grouping columns), measures (single-aggregate expressions), and optional filters. The agent generates metric views automatically after the physical model is deployed. Every domain gets at least one metric view. The measures are single aggregates by design. `SUM(revenue)`, `COUNT(DISTINCT customer_id)`, `AVG(order_value)`. Nested aggregates like `AVG(SUM(...))` are not supported in metric view YAML and the agent prevents them by construction.

If the LLM proposes a measure that references a column absent from the valid column list, the measure is dropped rather than quietly substituted. If a whole metric view ends up with no valid measures, a Row Count floor is emitted so the view is still installable and discoverable.

Metric views are the bridge between the Silver-layer Base Model and the Gold-layer analytics. They let BI tools like Databricks AI/BI, Tableau, and Power BI consume consistent KPI definitions without every team reinventing them.

Sample Data

Optionally, the agent generates synthetic sample data.

The engine is pool-based. For every table, pools of plausible values are generated up front (names, addresses, realistic monetary amounts, ISO country codes, email formats). Rows are then assembled by drawing from the pools. Foreign keys are respected deterministically. Every child row's FK points to a valid parent row. Regex constraints on columns are satisfied. Country codes are three letters. No Lorem Ipsum.

The widget controls how many rows per table (zero to one hundred). Zero skips generation entirely. Ten is a good default for review.

The 19 Pipeline Stages

The agent runs nineteen pipeline stages end to end. The durations below are typical, not guaranteed.

#	Stage	What Happens
1	Setup and Configuration	Validates inputs, creates metamodel tables
2	Interpreting Instructions	(Vibe mode only) Parses vibes into a structured action plan
3	Collecting Business Context	LLM enrichment of the user description
4	Designing Domains	Four-proposal ensemble plus judge

5	Creating Data Products	Per-domain tables, with per-domain architect review
6	Global Architect Review	Cross-domain structural review
7	Enriching Data Products with Attributes	Every column on every table
8	Cross-Domain Linking	In-domain plus pairwise plus global sweep
9	Quality Assurance	Nine sub-checks and auto-remediation
10	Applying Naming Conventions	Final naming consistency pass
11	Model Finalization	Logical snapshot before deploy
12	Subdomain Allocation	Products grouped into subdomains
13	Physical Schema Construction	Unity Catalog schemas plus Delta tables
14	Applying Foreign Keys	FK constraints in dependency order
15	Applying Tags	Classification and PII tags
16	Applying Metric Views	Databricks metric views
17	Generating Sample Data	Pool-based synthetic records
18	Generating Artifacts	README, Excel, DBML, RDF, data dictionary
19	Consolidation and Cleanup	Final pass over metadata

Stages 13-17 only run when a deployment catalog is supplied. The logical-only mode runs stages 1-12 and 18-19 and produces the `model.json` plus artifacts.

What You Actually Get

Every run produces the following artifacts.

Logical artifacts (always).

- `model.json`. The complete model as a single JSON file. This is the primary interchange format.
- `readme.md`. A human-readable description of the model.
- `vibes/current_vibes.txt` and `vibes/next_vibes.txt`. The input vibes and the recommended next-step vibes.
- `domains.json`, `products.json`, `attributes.json`. Each level of the model as its own file for tools that prefer flat structures.
- Excel or CSV exports under `docs/`.
- SQL DDL files under `schemas/`. One per domain plus a cross-domain FK file.

- DBML schema diagram under `diagram/`.
- RDF/Turtle ontology under `ontology/`.
- Auto-generated release notes.

Physical artifacts (when a deployment catalog is set).

- Unity Catalog schemas, with naming from the configured catalog style.
- Delta tables for every product.
- Foreign key constraints between tables.
- Unity Catalog tags on schemas, tables, and columns.
- Databricks metric views for KPI calculation.
- Synthetic sample data with valid FK references.

Where the Approach Is Honestly Strong

- **Speed.** Silver-layer models that used to take months take hours or days.
- **Specificity.** The model uses your terms, your domains, your divisions, your regulatory context.
- **Rule coverage.** More than two hundred enforceable rules means the output is consistent at a level most hand-written models never reach.
- **Governance.** Every column with sensitive data gets classified and tagged. Every PK/FK is named by a single convention. Every cataloging style is reproducible.
- **Evolvability.** Every new iteration is a new version, so the cost of change is close to zero.
- **Dual shape.** The same model is a relational schema, a DBML diagram, a knowledge-graph ontology, and a physical Unity Catalog deployment.
- **Logical-physical separation.** One `model.json`, three cataloging styles. Redeploy to a new environment or governance shape with zero rework.

Things to consider

- **Domain expertise still matters.** The agent can generate a very good first cut. Expert review still improves the model in ways only a practitioner would see. Vibe modeling shortens the loop but does not remove the expert.
- **Very narrow niches.** The LLM knows less about highly specialized sub-verticals than it does about mainstream industries. Expect to spend more vibes making a niche model idiomatic.
- **Non-deterministic.** Two base-model runs from the same input will produce closely similar but not identical outputs. This is by design (multi-proposal ensemble) and the vibe-iteration loop is the intended way to converge on a specific desired shape.
- **Governance acceptance.** Organizations with strict data-model approval committees still need to run the output through review. The agent produces the artifact faster. It does not remove the approval step.

We think these tradeoffs are the right ones. A model that takes a week to produce and is specific to the business is a better starting point than a model that takes a year to adapt and is the average of the sector.

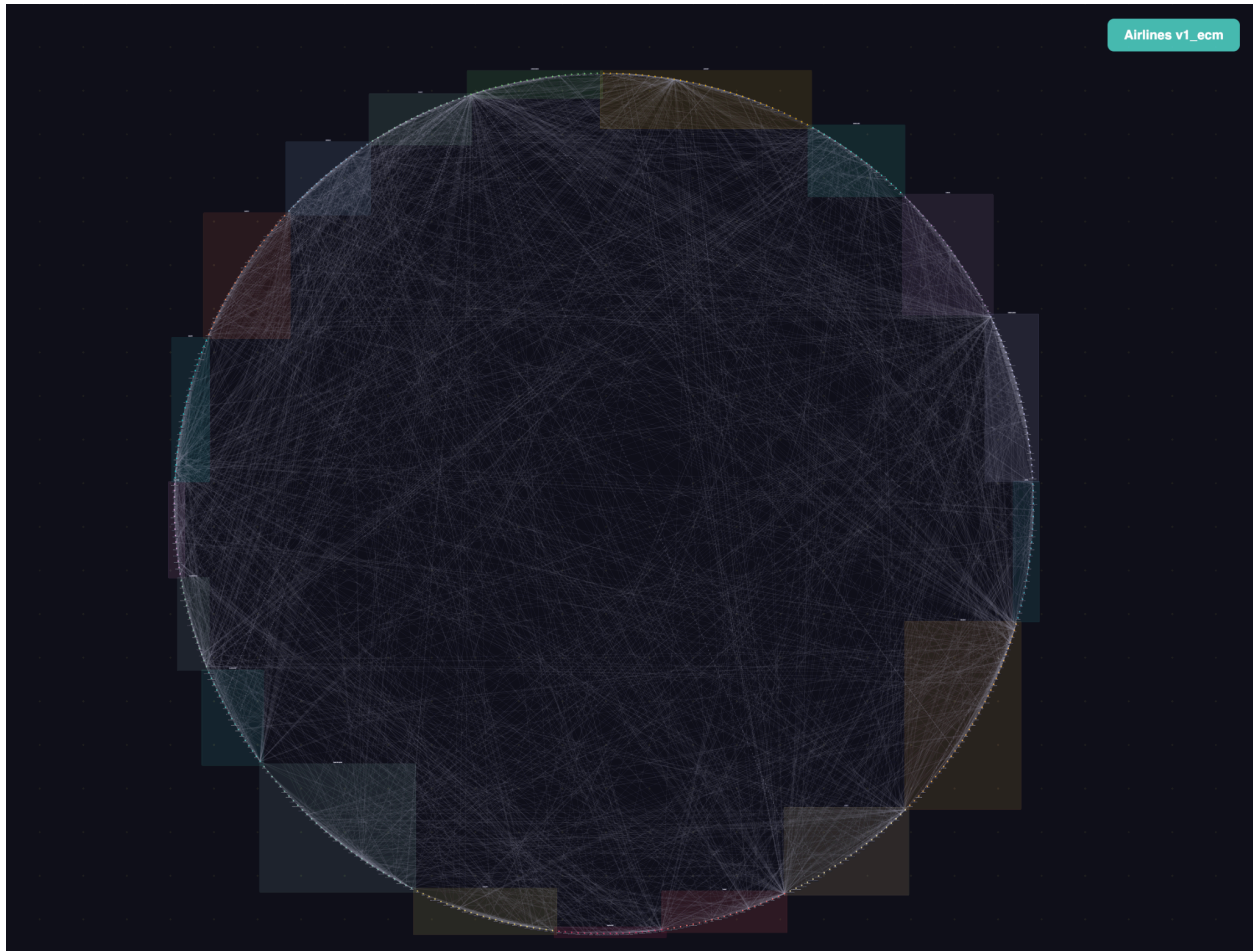
Example Vibed Model - Airline Data Model

We generated an ECM data model for Airlines that covers all business divisions. Below are the model metrics:

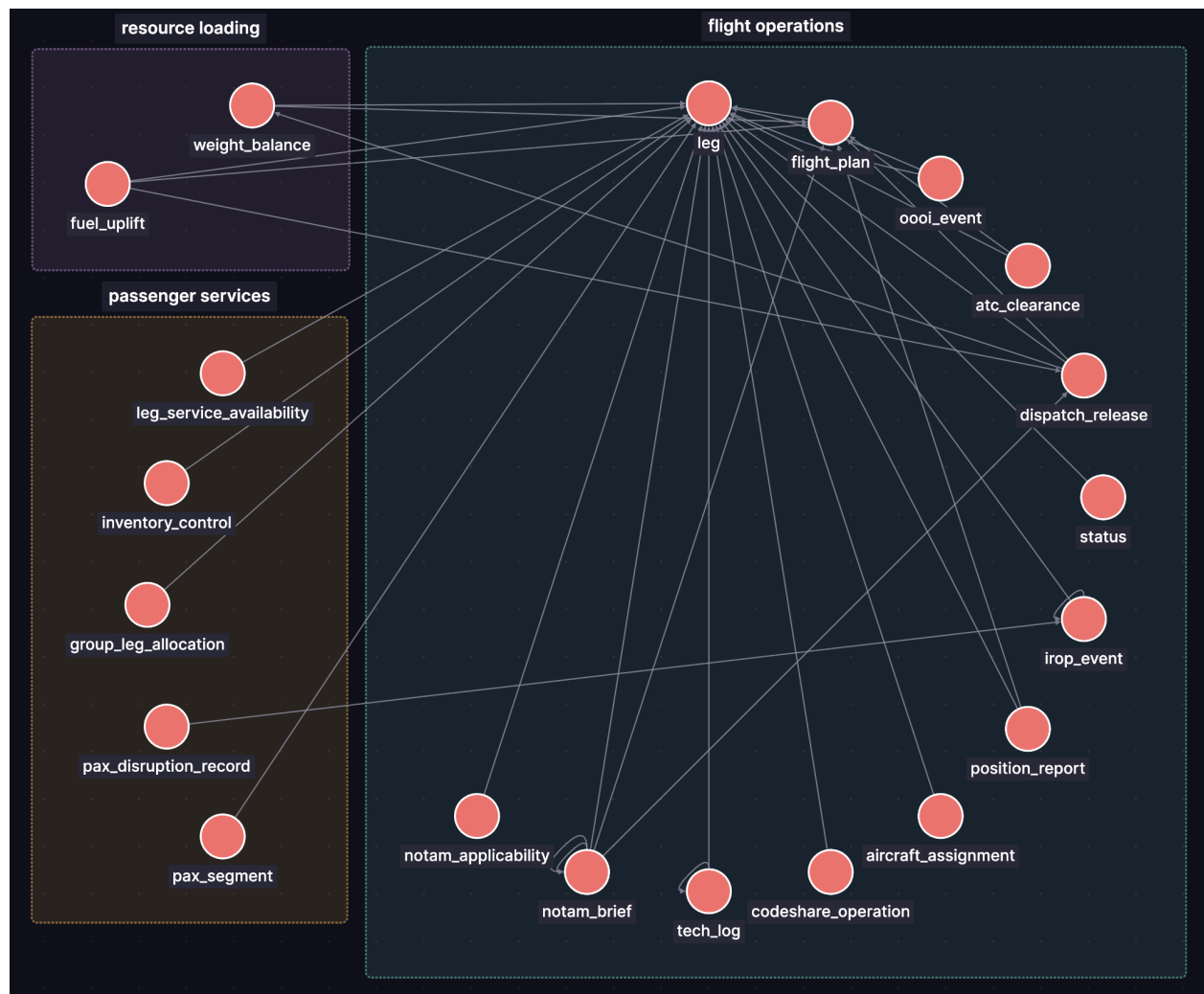
Model Metrics

Metric	Value
Model Scope	ECM (Expanded Coverage Model)
Total Domains	19
Total Subdomains	60
Total Products	420
Total Attributes	17278
Primary Keys	420
Foreign Keys	2877
Avg Attributes/Product	41.1
Metric Views	203

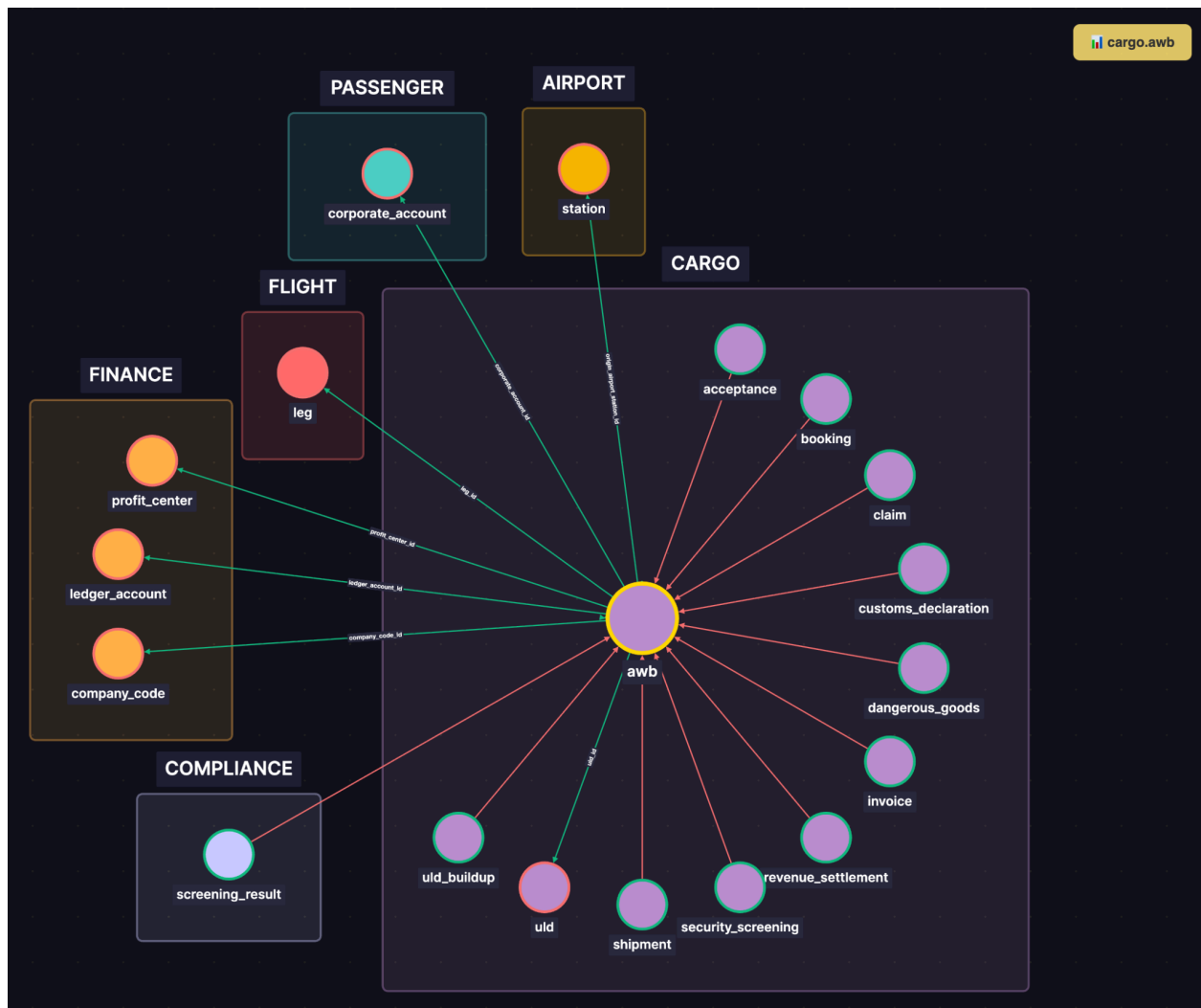
When you visualize this model, it will look like a fully connected DAG like below:



And this is how the flight domain looks:



When you drill down to, say, the Airway bill data product (awb), it looks like below:



Closing

A data model is a claim. It claims: this is what this business is. These are its concepts. These are their relationships. This is where each piece of data belongs.

Vibe Data Modeling makes that claim in two steps.

Step one plants the seed. It decomposes the organization into divisions, divisions into domains, domains into subdomains, subdomains into products, products into attributes. Then it links, validates, deploys, and self-assesses. The output is version zero.

Step two iterates on that seed through natural language. Understanding the modeler's intent, executing with precision, validating with the same rigor as the original build. Each vibe produces a new version. Each closer to the ideal.

The result is a Silver-layer model that is both technically correct and organizationally owned. Every team has had their say. Every domain reflects the language its users actually speak. Every table exists because someone needed it.

The model is not a document filed away in a drawer. It is a living, versioned, evolvable structure, deployed to a lakehouse, governed with classification tags, expressed as both a relational schema and a semantic ontology, designed from birth to grow.

It is not perfect. No model ever is. But it is honest about its imperfections, and it tells its creator exactly where to look next.

Appendix A: Rule Catalog Summary

The full rule catalog is published at `docs/whitepaper.md` in the repository. A summary of groups appears below.

Group	Name	Scope
G01	Naming Convention Rules	How to name domains, tables, columns consistently.
G02	Semantic Deduplication Rules	How to detect and resolve duplicate concepts.
G03	Foreign Key Rules	How to create and maintain FK relationships.
G04	Primary Key Rules	How to define and manage primary keys.
G05	Normalization Rules	3NF enforcement and denormalization policy.
G06	Domain and Division Rules	How to organize domains and divisions.
G07	Data Type Rules	Approved Spark SQL types and inference.
G08	Tag and Classification Rules	Data sensitivity classification and tagging.
G09	Relationship and Graph Rules	DAG enforcement, cycle breaking, no silos.
G10	Sample Data Rules	Record counts, FK integrity in samples.
G11	Quality and Validation Rules	Range checks and min/max enforcement.
G12	Product Design Rules	Table design, M:N relationships, core/helper.
G13	Sample Data QA Rules	Sample-data correctness gates.
G14	Vibe Constraints Rules	Mutation budgets and vibe-mode governance.
G15	Physical Schema Deployment	UC deployment, FK ordering, consolidation.
SUB	Subdomain Rules	Subdomain sizing, naming, balance.

Appendix B: Getting Started

The reference implementation is a single Databricks notebook at `agent/dbx_vibe_modelling_agent.ipynb` in the repo. Fill the four core widgets (Business name, Description, Model Scope, Deployment Catalog) and run. Everything else auto-fills.

For producing both MVM and ECM in one go with staging and test install, use the `runner/vibe_runner.ipynb` orchestrator. It runs a four-task Databricks Job: ECM generate, ECM install, MVM shrink, MVM install.

The repo also contains a test notebook at `tests/vibe_tester.ipynb` that exercises every operation across a tiny business, a mid-size business, and an industry-scale business.

The Vibe Modelling Agent is built on Databricks Serverless Compute with Unity Catalog governance. It is provided as-is, through the repository, and is not officially supported through Databricks customer support channels. Issues, questions, and feature requests go through the repository's Issues tab.